Handoff — 2026-08-04 (overnight)

Branch `main`, pushed. Suite 58 passed, 0 failed throughout.

Read `docs/ROLLOUT-design.md` first — it is where the session ended and the only thing with a live decision attached. `docs/DATA-LAW.md` is new and short, and it is the rule Will asked for rather than the habit we had.

---

1. THE SHORT VERSION

One real bug fixed. One headline number cut in half and made honest. One metric built and thrown away for being wrong. One cross-language head-to-head built and **withdrawn** for failing its own alignment check. One engine taken from 89.2% to 98.5%. And one result nobody was looking for, which is the largest thing here.

the `Infinite loop` in `lookahead_cost.js`	**fixed** — 0 failures in 8,000 forks
"one turn of foresight is worth +4.91"	**+2.29** after removing the clock
MEDICHAM click coverage	**89.2% -> 98.5%**
a rollout leaf, cost per call	**5.83 ms** — what ONE forked Showdown turn costs
greedy playout -> RANDOM playout	**64.42% -> 68.13%**

Do not quote a "rollout vs PORYGON2 head to head". One was produced (+1.83, CI +0.11 to +3.53, printed as "R1 PASSES") and it is withdrawn — §5.4. It was scoring the two judges on different turns.

The live number is the same-corpus one: at `explore=1`, the rollout beats material by **+2.82 points, 95% CI −0.06 to +5.69**, against PORYGON2's published +3.42 over that baseline. A larger run is in flight to see whether that interval clears zero.

---

2. THE FORK BUG, AND WHY THE HYPOTHESIS WAS WRONG

`Infinite loop` is **not** a turn-loop guard. `sim/battle.ts:584` throws it when `log.length - sentLogPos > 1000` — a log-length guard. And `sim/state.ts:72` does `state.log = battle.log`, aliasing rather than copying, with `:153` assigning it straight back.

So every fork restored from one snapshot appended to **one shared array**. ~38 forks to the threshold, then every fork after it throws. That is why the rates were 40/40, 27/40, 30/40, 24/40, 25/40 — a contiguous tail tracking lines-per-turn, nothing to do with the board. The tell was in the original output: fork **33** threw, not fork 1.

The recorded hypothesis — a restored battle is not in `requestState === 'move'` — was wrong. `requestState` is `"move"` and the turn advances. The driving was always fine.

The consequence that never threw is worse: the snapshot aliases the *live* battle's log, so a search forking the game it is playing writes simulated events into that game's record.

Fixed as `CS.snapshot()` / `CS.forkBattle()`. The pinned checkout is not edited (ADR-001).

---

3. THE CLOCK, WHICH TAKES +4.91 DOWN TO +2.29

`engine/lookahead_bound.py` compares two columns that are **different time slices of the same games** — states `0..L-1-H` against states `H..L-1`, same labels. Later positions are easier to classify because the game is more decided. That alone produces a positive "oracle gain" with no lookahead information.

`engine/lookahead_clock_control.py` predicts the gain from the turn-index shift alone. Accuracy by index runs **50.88% at turn 0 to 73.74% at turn 9**.

```
one turn    +4.93 observed  ->  2.65 clock  ->  +2.29 real
two turns   +9.07 observed  ->  5.38 clock  ->  +3.69 real
```

`+4.93` reproduces the `+4.91` on record, so this is the same instrument. **Raw, the second step looked bigger than the first (+5.13 vs +3.84); controlled, the ordering reverses (+1.40 vs +2.29).** The clock is why depth looked free.

A search cannot collect the clock part: its candidate successors all sit at the same index, so the advantage applies to every one and cancels out of the ranking — and ranking is all a search does.

Controlling for index-from-START is right; index-from-END would over-control, because a search *can* pick lines that end sooner in its favour.

---

4. MEDICHAM, 89.2% -> 98.2%

Will's question started it: *"cant it just be all the same calcs we already do"*. `battleInit` never reset HP, so MEDICHAM could always have been seeded mid-game — nothing ever did it.

Each mechanic derived from the tags artifact, verified, committed separately:

added	gain
Trick Room (all the state existed; nothing could SET it)	+1.18
heal family, from the rulebook's own fraction	+0.80
redirection, from the `redirects` tag Will remembered was there	+1.78
screens, from `halvesDamage` {mult, category}	+1.69
switching — voluntary, pivot, faint, through one path	+2.12
weather setters	+0.26
Helping Hand	+0.67
ally boosts, Perish Song, Yawn	+0.52

Will's corrections, all of which landed in the code: Aurora Veil needs snow (the tag says `needsWeather: true`, a boolean — it records THAT, not WHICH); Brick Break / Psychic Fangs / Raging Bull break screens, before damage, so the breaking hit is itself unreduced; Light Clay extends them; and **Prankster's +1 was modelled nowhere**, so a Prankster screen went up *after* the attack it was meant to blunt.

A bug that was always there, exposed only once switching existed: moves targeted a Pokemon *object*, so a mon that switched out stayed targetable from the bench. Moves target a **slot**.

Still not modelled, deliberately: `lowersTarget` is 22 moves including Charm and Fake Tears, and the tag says `readFrom: "m.boosts"` — the table lives on the dex move object, which this browser bundle cannot reach, and `data/move-effects.js` is generated from **CHOMP's** json, a separate repo. Making it derivable is cross-repo work, not a hardcode here.

---

5. THE ROLLOUT LEAF, AND WHAT R1 SAYS

`docs/ROLLOUT-design.md`. The premise: `data/porygon2c.json` scores PORYGON2 at 63.70% against 60.28% for material sign, so **the entire learned value function is worth 3.4 points over counting bodies**. A search maximising it is close to a search for "take the most material", which greedy already does.

5.1 The literature says the design is contested

Foul Play — top 100 Gen9 OU, #1 RandomBattleBlitz, 71–88% GXE — runs MCTS *"guided by a custom evaluation function rather than rollouts"*, and abandoned expectiminimax past ~5 turns. But Will asked the right question: **it plays singles.** One action from ~9 there against a pair from ~64 joint here, and redirection, spread damage and Wide Guard do not exist in singles at all.

Also on record: truncated rollouts cut cost *and* variance (Tesauro), which MEDICHAM already did; and a **stronger playout policy does not reliably make search stronger**, which constrains the obvious follow-up.

5.2 THE RESULT OF THE NIGHT: a random playout judges better than a greedy one

`chooseAction` is deterministic greedy, so every playout from a position replays the same line and the N samples agree with **each other** rather than with reality. The calibration table shows it saturating: it says 0-10% on 2,245 positions that actually win **26.0%**, and 90-100% on 2,612 that actually win **75.9%**. 53% of positions land in those two bins.

§3.2 of the design doc had recorded the fix before it was needed — *heavy rollouts help only when they avoid becoming low-variance*. Same 1,456 positions, n=40, `explore` = probability of clicking a random legal move instead of the greedy one:

```
explore=0      64.42%   Brier 0.2647   logloss 1.7400
explore=0.35   65.11%          0.2391           1.0691
explore=0.6    65.73%          0.2265           0.9293
explore=1.0    68.13%          0.2192           0.8849
```

Monotone in every column. A fully random playout is the best judge — +3.71 over greedy, +3.29 over material, against PORYGON2's published +3.42 over that baseline. Making the simulated players *worse* made the estimate *better*.

The randomness is in the rollout harness, not in `chooseAction`, which is the Tower's and the live bot's policy.

5.3 N is not the lever (at explore=0), and becomes one (at explore=1)

Paired on the same 4,487 positions, one replay:

```
n=10    64.03%   Brier 0.2739
n=40    64.68%   Brier 0.2657
n=160   64.48%   Brier 0.2645
```

Flat. Sixteen times the compute moves accuracy under half a point, not monotonically. So the leaf is **bias-limited**: more samples converge on the bias, and the suspect is `chooseAction` or engine fidelity. It also means the leaf is ~10x cheaper than budgeted.

5.4 The head-to-head, and the corpus trap it nearly walked into

The first join returned **zero** rows. Not a bug: `P.HUMAN_RAW` is `games.ladder.raw-logs.jsonl`, while `fit_policy.loadCorpus()` feeds from three stores and its first 1,200 games are dominated by **bo3**. 1,200 game ids against 29,580 ladder games, no overlap at all.

So PORYGON2's published 63.70% is a ladder-only figure, and the corpus it is quoted beside is 54.7% bo3. Every comparison drawn against it — including mine earlier in the session — was across different populations.

Joined across both raw stores, 2,566 identical positions:

```
material (porygon2 form)  60.21%   Brier 0.2276
PORYGON2 k=200            59.63%   Brier 0.2291
ROLLOUT                   61.46%   Brier 0.2878

rollout - PORYGON2   +1.83 points   95% CI -0.48 to +4.15   UNDECIDED
```

PORYGON2 lands below material here (−0.58) where it publishes +3.42. Population, not a bug — but its headline does not replicate off ladder.

Weakness, stated: 1,921 of 4,487 rows were dropped by the alignment witness, and the witness is `alive_diff`, which is 0 on most early turns and so passes trivially there. A continuous HP witness has been added and is *reported before it filters*, because a witness that rejects everything on a definitional difference looks identical to a genuinely misaligned join.

---

6. WHAT I THREW AWAY, AND WHY

A partial-coverage metric that reported **"honest fidelity 51.1%, not 94.6%"**. Its three largest entries were `stalling` (Protect), `doublesSideSpeed` (Tailwind) and `reversesSpeed` (Trick Room) — all implemented, the last verified by test the same day. It matched on **tag names**, and MEDICHAM implements mechanics without naming tags.

The artifact cannot answer it either: `consumedBy` names a **board.js feature** (`stallingMove`, `speedSide`, `trickRoomField`), so `used: true` means MAG has a feature for it, not that the rollout simulates it.

What shipped instead is the one figure verified by reading the engine: pivot moves are 2.99% of clicks, and 0.87% of all clicks were pivots counted as fully-modelled attacks. **0.87 points of overstatement, not 43.5.**

6.5 THE GATES, AS THEY STAND

```
R1  the rollout is a better JUDGE     PASSES ON THE BASELINE
    68.18% vs material 65.26% on 9,201 positions, +2.91 [1.79, 4.04].
    PORYGON2's published +3.42 sits INSIDE that interval, so the rollout is at
    least its equal and the two are not separated. Not the same as beating it.

R2  it is affordable                  PASSES
    5.83 ms per leaf at n=10. A Showdown fork is 4.52 ms and buys ONE turn.
    K=3 is 0.47 s, K=4 is 1.49 s median. Cost will not be what kills this.

R3  the pick changes                  PASSES
    72.4% divergence against a 43.8% self-disagreement floor, 185 decisions.
    The floor is the caveat, not a footnote — see below.

R4  it wins games                     NOT STARTED
```

R4 is the only one that matters and it is not built. Everything above says the leaf is better and the player is different; none of it says the different player wins. That needs the rollout search wired into an actual player and an SPRT against shipped greedy.

Before R4, fix the instability. The search disagrees with ITSELF on 43.8% of decisions at n=200. An SPRT against a player that unstable will be noisy and slow to decide. Raising N is the lever and R2 says it is cheap — n=200 costs ~110 ms per leaf, and a decision at K=3 is 9 leaves.

---

7. WHERE TO START

1. **R4.** Wire the rollout search into a player and SPRT it against shipped greedy. It is the only gate that answers the question, and R1/R2/R3 exist to justify paying for it.
2. **Drop the self-disagreement first** (§6.5). Raise N until the search is stable enough that an SPRT is not fighting its own noise.
3. **The remaining bias.** §5.2's sweep is monotone all the way to explore=1, so a fully random playout may not be the end of it — the next question is whether a *biased-random* playout (weight by move power, or by MAG's prior) beats uniform. Note §5.1's warning and the fact that greedy was the WORST of the six settings: measure, do not assume more knowledge helps.
4. `screenValue` **in board.js** overstates screens by ~1.5x — it uses the tag's singles 0.5 where doubles is 0.667. Second-order because a fitted weight absorbs a scale error, EXCEPT the feature is clipped at 1, so saturation is real. Costs a refit plus seven artifacts; measure how often it saturates first.

Do not re-inherit: +4.91 (it is +2.29), PORYGON2's +3.42 over material (ladder-only), and the idea that more rollouts will help (flat in N).